



## **CENTRO UNIVERSITÁRIO IESB**

Graduação em Ciência de Dados e Inteligência Artificial

# **RELATÓRIO TÉCNICO**

Projeto de IAGEN: Chatbot com IA Generativa (Amazon Bedrock) para Consulta em Linguagem Natural à Base do INEP/Censo Escolar

Autor: Eduardo Gonçalves Barbosa

Docente: Sérgio da Costa Côrtes

Disciplina: Integração de Técnicas e Projeto de Sistemas Inteligentes

\*

## RESUMO

Este relatório técnico apresenta o desenvolvimento de um *chatbot* acadêmico capaz de responder, em linguagem natural, perguntas sobre a base educacional do INEP/Censo Escolar. A solução utiliza o serviço de IA generativa **Amazon Bedrock** (modelo *LLaMA 4 Scout*) exclusivamente como **classificador de intenções**, devolvendo um JSON controlado, enquanto a execução das consultas é realizada por **funções Python seguras e somente leitura** sobre um banco **PostgreSQL**. O sistema inspeciona a estrutura real das tabelas, descobre os relacionamentos a partir das chaves estrangeiras (sem inventar chaves) e consolida o escopo definido pelo docente em uma consulta/visão denominada **vw\_escopo\_ia**. A interface é construída em **Streamlit**. São discutidas a arquitetura, as decisões de segurança contra injeção de SQL, as dificuldades encontradas e os resultados obtidos com dados reais do Censo Escolar de 2025.

**Palavras-chave:** chatbot. inteligência artificial generativa. amazon bedrock. censo escolar. linguagem natural. postgresql.

# SUMÁRIO

|            |                                                                          |           |
|------------|--------------------------------------------------------------------------|-----------|
| <b>1</b>   | <b>INTRODUÇÃO . . . . .</b>                                              | <b>4</b>  |
| <b>1.1</b> | <b>Objetivo Geral . . . . .</b>                                          | <b>4</b>  |
| <b>1.2</b> | <b>Objetivos Específicos . . . . .</b>                                   | <b>4</b>  |
| <b>1.3</b> | <b>Justificativa . . . . .</b>                                           | <b>5</b>  |
| <b>2</b>   | <b>FUNDAMENTAÇÃO TEÓRICA . . . . .</b>                                   | <b>6</b>  |
| <b>2.1</b> | <b>Chatbots . . . . .</b>                                                | <b>6</b>  |
| <b>2.2</b> | <b>Inteligência Artificial Generativa . . . . .</b>                      | <b>6</b>  |
| <b>2.3</b> | <b>Amazon Bedrock . . . . .</b>                                          | <b>6</b>  |
| <b>2.4</b> | <b>Bancos de Dados Relacionais e Modelagem Dimensional . . . . .</b>     | <b>6</b>  |
| <b>2.5</b> | <b>Linguagem Natural Aplicada à Consulta de Dados (NL2SQL) . . . . .</b> | <b>6</b>  |
| <b>2.6</b> | <b>Dados Educacionais: INEP e Censo Escolar . . . . .</b>                | <b>7</b>  |
| <b>2.7</b> | <b>Segurança na Geração de Consultas . . . . .</b>                       | <b>7</b>  |
| <b>3</b>   | <b>ESCOPO DO PROJETO E METODOLOGIA . . . . .</b>                         | <b>8</b>  |
| <b>3.1</b> | <b>Metodologia . . . . .</b>                                             | <b>8</b>  |
| <b>3.2</b> | <b>Escopo Definido pelo Professor . . . . .</b>                          | <b>8</b>  |
| <b>3.3</b> | <b>Tabelas Utilizadas . . . . .</b>                                      | <b>8</b>  |
| <b>3.4</b> | <b>Variáveis Seleccionadas . . . . .</b>                                 | <b>8</b>  |
| <b>3.5</b> | <b>Processo de Junção das Tabelas . . . . .</b>                          | <b>9</b>  |
| 3.5.1      | Inspeção da Estrutura Real . . . . .                                     | 9         |
| 3.5.2      | Relacionamentos Identificados (Chaves Reais) . . . . .                   | 9         |
| <b>3.6</b> | <b>Criação da VIEW vw_escopo_ia . . . . .</b>                            | <b>10</b> |
| 3.6.1      | Visão Física x Visão Lógica . . . . .                                    | 10        |
| <b>4</b>   | <b>DESENVOLVIMENTO E ARQUITETURA . . . . .</b>                           | <b>12</b> |
| <b>4.1</b> | <b>Arquitetura do Sistema . . . . .</b>                                  | <b>12</b> |
| <b>4.2</b> | <b>Tecnologias Utilizadas . . . . .</b>                                  | <b>12</b> |
| <b>4.3</b> | <b>Integração com o Amazon Bedrock . . . . .</b>                         | <b>12</b> |
| <b>4.4</b> | <b>Conexão com o Banco de Dados . . . . .</b>                            | <b>13</b> |
| <b>4.5</b> | <b>Funções Implementadas . . . . .</b>                                   | <b>13</b> |
| <b>4.6</b> | <b>Consulta Livre em Linguagem Natural (NL→SQL) . . . . .</b>            | <b>14</b> |
| <b>4.7</b> | <b>Fluxo de Funcionamento do Chatbot . . . . .</b>                       | <b>14</b> |
| <b>5</b>   | <b>RESULTADOS E DISCUSSÃO . . . . .</b>                                  | <b>15</b> |
| <b>5.1</b> | <b>Prints da Execução . . . . .</b>                                      | <b>15</b> |

|     |                                             |    |
|-----|---------------------------------------------|----|
| 5.2 | Testes Realizados . . . . .                 | 16 |
| 5.3 | Exemplos de Perguntas e Respostas . . . . . | 16 |
| 5.4 | Dificuldades Encontradas . . . . .          | 16 |
| 5.5 | Resultados Obtidos . . . . .                | 17 |
| 5.6 | Limitações do Projeto . . . . .             | 17 |
| 5.7 | Possibilidades de Melhoria . . . . .        | 17 |
| 6   | CONCLUSÃO . . . . .                         | 18 |

|                       |    |
|-----------------------|----|
| REFERÊNCIAS . . . . . | 19 |
|-----------------------|----|

|                     |    |
|---------------------|----|
| APÊNDICES . . . . . | 20 |
|---------------------|----|

|     |                                                               |    |
|-----|---------------------------------------------------------------|----|
|     | APÊNDICE A – TRECHOS PRINCIPAIS DO CÓDIGO . . . . .           | 21 |
| A.1 | Descoberta de Relacionamentos (sem inventar chaves) . . . . . | 21 |
| A.2 | Guarda de Segurança (somente leitura) . . . . .               | 21 |
| A.3 | Agregação Segura de Matrículas por Grupo . . . . .            | 21 |

## LISTA DE ILUSTRAÇÕES

|                                                             |    |
|-------------------------------------------------------------|----|
| Figura 1 – Arquitetura e fluxo de dados do chatbot. . . . . | 12 |
| Figura 2 – Tela principal do chatbot. . . . .               | 15 |
| Figura 3 – Exemplo de resposta (matrículas por UF). . . . . | 15 |

## LISTA DE TABELAS

|                                                                                     |    |
|-------------------------------------------------------------------------------------|----|
| Tabela 1 – Variáveis do escopo e tabela de origem (após inspeção do banco). . . . . | 9  |
| Tabela 2 – Relacionamentos descobertos por introspecção. . . . .                    | 9  |
| Tabela 3 – Exemplos de perguntas e respostas obtidas com dados reais. . . . .       | 16 |

# 1 INTRODUÇÃO

A consulta a grandes bases de dados governamentais, como as do Instituto Nacional de Estudos e Pesquisas Educacionais Anísio Teixeira (INEP), normalmente exige conhecimento técnico de SQL e da modelagem das tabelas. Isso cria uma barreira para usuários não especialistas que desejam apenas obter informações como o total de matrículas por região ou a distribuição de estudantes por raça/cor.

Os avanços recentes em Inteligência Artificial Generativa permitem traduzir a intenção de perguntas escritas em linguagem natural para ações estruturadas. Este projeto emprega o Amazon Bedrock (Amazon Web Services, 2025b) para interpretar perguntas e um conjunto de funções Python seguras para executar as consultas, conciliando usabilidade (linguagem natural) e segurança (a IA nunca executa SQL).

## 1.1 OBJETIVO GERAL

Desenvolver um *chatbot* com IA generativa, integrado ao Amazon Bedrock e conectado a um banco de dados educacional, capaz de responder perguntas em linguagem natural sobre dados do INEP/Censo Escolar, respeitando o escopo de tabelas e variáveis definido pelo docente.

## 1.2 OBJETIVOS ESPECÍFICOS

1. Conectar o projeto Python ao Amazon Bedrock.
2. Conectar o projeto Python ao banco de dados PostgreSQL.
3. Identificar, por introspecção, a estrutura real das tabelas indicadas.
4. Realizar a junção entre as tabelas `inep_censo_escolar`, `municipio`, `unidade_federacao` e `regiao`.
5. Criar/validar uma consulta consolidada (visão) `vw_escopo_ia`.
6. Permitir perguntas em linguagem natural e interpretá-las via Bedrock.
7. Executar consultas seguras (somente leitura) e retornar respostas claras.
8. Construir uma interface simples em Streamlit.
9. Documentar o projeto neste relatório técnico.

## 1.3 JUSTIFICATIVA

O projeto justifica-se por três frentes. Do ponto de vista **educacional**, democratiza o acesso a indicadores do Censo Escolar para gestores, pesquisadores e estudantes sem exigir domínio de SQL. Do ponto de vista **técnico**, demonstra um padrão arquitetural seguro de integração entre Modelos de Linguagem (LLMs) e bancos de dados, no qual o modelo apenas classifica a intenção e o código executa consultas validadas, mitigando riscos de injeção de SQL. Do ponto de vista **acadêmico**, exercita conceitos de IA generativa, modelagem dimensional, introspecção de esquemas e boas práticas de engenharia de software.

## 2 FUNDAMENTAÇÃO TEÓRICA

### 2.1 CHATBOTS

*Chatbots* são sistemas que conversam com usuários em linguagem natural. Os de geração mais recente apoiam-se em LLMs para compreender intenção e gerar respostas, superando abordagens baseadas apenas em regras fixas.

### 2.2 INTELIGÊNCIA ARTIFICIAL GENERATIVA

A IA generativa baseia-se em redes *Transformer* (VASWANI et al., 2017), capazes de modelar dependências de longo alcance em texto. LLMs como a família *LLaMA* (Meta AI, 2025) são treinados para prever o próximo *token* e, com instruções adequadas (*prompt engineering*), podem ser direcionados a tarefas específicas, como a classificação de intenções.

### 2.3 AMAZON BEDROCK

O Amazon Bedrock (Amazon Web Services, 2025b) é um serviço gerenciado da AWS que oferece acesso, via API, a modelos de fundação de diversos provedores. Neste projeto utilizou-se o modelo `us.meta.llama4-scout-17b-instruct-v1:0` por meio da *Converse API* (Amazon Web Services, 2025a), com *fallback* para `invoke_model`. O acesso é autenticado pelo AWS IAM Identity Center (SSO).

### 2.4 BANCOS DE DADOS RELACIONAIS E MODELAGEM DIMENSIONAL

Bancos relacionais organizam dados em tabelas relacionadas por chaves primárias e estrangeiras. A modelagem dimensional separa **tabelas-fato** (medidas, como a quantidade de matrículas) de **tabelas-dimensão** (atributos descritivos, como o nome do município). Esse padrão orienta a junção realizada neste trabalho.

### 2.5 LINGUAGEM NATURAL APLICADA À CONSULTA DE DADOS (NL2SQL)

A tradução de linguagem natural para consultas estruturadas é tema clássico de pesquisa (ZHONG; XIONG; SOCHER, 2017; YU et al., 2018). Em vez de gerar SQL

livremente com o LLM (o que é arriscado), adotou-se uma estratégia de classificação de intenção: o modelo escolhe uma ação dentre um conjunto fixo e seguro, e o SQL é montado por código auditável.

## 2.6 DADOS EDUCACIONAIS: INEP E CENSO ESCOLAR

O Censo Escolar (Instituto Nacional de Estudos e Pesquisas Educacionais Anísio Teixeira (INEP), 2025) é o principal levantamento estatístico da educação básica brasileira, coletado anualmente pelo INEP. Contém variáveis sobre escolas, matrículas, docentes e infraestrutura, servindo de base para políticas públicas e indicadores educacionais.

## 2.7 SEGURANÇA NA GERAÇÃO DE CONSULTAS

A injeção de SQL figura entre as principais vulnerabilidades de aplicações (OWASP Foundation, 2021). As defesas adotadas neste trabalho incluem: separação entre interpretação (IA) e execução (código), consultas parametrizadas, *allowlist* de tabelas, validação de identificadores contra o esquema real e transações somente leitura.



## 3 ESCOPO DO PROJETO E METODOLOGIA

### 3.1 METODOLOGIA

O desenvolvimento seguiu as etapas: (i) configuração do acesso ao Amazon Bedrock e ao banco; (ii) inspeção da estrutura real das tabelas do escopo; (iii) descoberta dos relacionamentos por chaves estrangeiras; (iv) construção da consulta consolidada `vw_escopo_ia`; (v) implementação das funções seguras de consulta; (vi) integração com o Bedrock como classificador de intenções; (vii) construção da interface em Streamlit; e (viii) testes automatizados e validação com dados reais.

### 3.2 ESCOPO DEFINIDO PELO PROFESSOR

O escopo do trabalho foi definido pelo docente da disciplina, que indicou as tabelas e as variáveis a serem utilizadas. **Toda a modelagem deste projeto seguiu estritamente esse escopo.**

### 3.3 TABELAS UTILIZADAS

- `inep_censo_escolar` — tabela principal (fato);
- `municipio` — dimensão (nome do município);
- `unidade_federacao` — dimensão (nome/sigla da UF);
- `regiao` — dimensão (nome da região);
- `inep_censo_escolar_matricula` — tabela-detalle das matrículas, identificada durante a inspeção (Seção 3.5).

### 3.4 VARIÁVEIS SELECIONADAS

As variáveis priorizadas pelo docente são apresentadas na Tabela 1.

Tabela 1 – Variáveis do escopo e tabela de origem (após inspeção do banco).

| Variável                    | Descrição                     | Tabela de origem   |
|-----------------------------|-------------------------------|--------------------|
| TP_CATEGORIA_ESCOLA_PRIVADA | Categoria da escola privada   | inep_censo_escolar |
| TP_LOCALIZACAO              | Localização (urbana/rural)    | inep_censo_escolar |
| TP_LOCALIZACAO_DIFERENCIADA | Localização diferenciada      | inep_censo_escolar |
| QT_MAT_INF                  | Matrículas - Ed. Infantil     | inep_censo_escolar |
| QT_MAT_FUND                 | Matrículas - Ens. Fundamental | inep_censo_escolar |
| QT_MAT_MED                  | Matrículas - Ens. Médio       | inep_censo_escolar |
| QT_MAT_BAS_FEM              | Matrículas femininas          | inep_censo_escolar |
| QT_MAT_BAS_MASC             | Matrículas masculinas         | inep_censo_escolar |
| QT_MAT_BAS_ND               | Raça/cor não declarada        | inep_censo_escolar |
| QT_MAT_BAS_BRANCA           | Raça/cor branca               | inep_censo_escolar |
| QT_MAT_BAS_PRETA            | Raça/cor preta                | inep_censo_escolar |
| QT_MAT_BAS_PARDA            | Raça/cor parda                | inep_censo_escolar |
| QT_MAT_BAS_AMARELA          | Raça/cor amarela              | inep_censo_escolar |
| QT_MAT_BAS_INDIGENA         | Raça/cor indígena             | inep_censo_escolar |

## 3.5 PROCESSO DE JUNÇÃO DAS TABELAS

### 3.5.1 Inspeção da Estrutura Real

Antes de qualquer junção, o sistema inspecionou a estrutura real do banco (PostgreSQL 17.4, *host* bigdata.dataiesb.com, base iesb, esquema public). A inspeção revelou um achado importante: as variáveis de matrícula (QT\_MAT\_\*) não estão em *inep\_censo\_escolar* (302 colunas), mas na tabela-detalle *inep\_censo\_escolar\_matricula* (237 colunas), relacionada à fato por (nu\_ano\_censo, co\_entidade). Já as variáveis TP\_\* estão na própria tabela *inep\_censo\_escolar*. Por isso, a tabela de matrículas foi incorporada à junção, para que o escopo do professor fosse plenamente atendido.

### 3.5.2 Relacionamentos Identificados (Chaves Reais)

Os relacionamentos foram descobertos a partir das chaves estrangeiras declaradas no banco, sem inventar nomes. A Tabela 2 resume as ligações.

Tabela 2 – Relacionamentos descobertos por introspecção.

| Origem                       | Coluna(s)                   | Destino                       |
|------------------------------|-----------------------------|-------------------------------|
| inep_censo_escolar_matricula | (nu_ano_censo, co_entidade) | inep_censo_escolar            |
| inep_censo_escolar           | co_municipio                | municipio.codigo_municipio_dv |
| municipio                    | cd_uf                       | unidade_federacao.cd_uf       |
| unidade_federacao            | cd_regiao                   | regiao.cd_regiao              |

## 3.6 CRIAÇÃO DA VIEW VW\_ESCOPO\_IA

A consulta consolidada reúne o nome do município, da UF e da região, além das variáveis do escopo. O Código 3.1 mostra o SELECT gerado automaticamente pelo sistema.

Listing 3.1 – Definição da visão vw\_escopo\_ia.

```

1 CREATE OR REPLACE VIEW vw_escopo_ia AS
2 SELECT
3     m.nome_municipio          AS nome_municipio ,
4     u.sigla_uf                AS sigla_uf ,
5     u.nome_uf                 AS nome_uf ,
6     r.nome_regiao             AS nome_regiao ,
7     f.tp_categoria_escola_privada AS tp_categoria_escola_privada ,
8     f.tp_localizacao          AS tp_localizacao ,
9     f.tp_localizacao_diferenciada AS tp_localizacao_diferenciada ,
10    mat.qt_mat_inf             AS qt_mat_inf ,
11    mat.qt_mat_fund            AS qt_mat_fund ,
12    mat.qt_mat_med             AS qt_mat_med ,
13    mat.qt_mat_bas_fem         AS qt_mat_bas_fem ,
14    mat.qt_mat_bas_masc        AS qt_mat_bas_masc ,
15    mat.qt_mat_bas_nd          AS qt_mat_bas_nd ,
16    mat.qt_mat_bas_branca      AS qt_mat_bas_branca ,
17    mat.qt_mat_bas_preta       AS qt_mat_bas_preta ,
18    mat.qt_mat_bas_parda       AS qt_mat_bas_parda ,
19    mat.qt_mat_bas_amarela     AS qt_mat_bas_amarela ,
20    mat.qt_mat_bas_indigena    AS qt_mat_bas_indigena
21 FROM inep_censo_escolar f
22 LEFT JOIN inep_censo_escolar_matricula mat
23     ON mat.nu_ano_censo = f.nu_ano_censo AND mat.co_entidade =
        f.co_entidade
24 LEFT JOIN municipio          m ON f.co_municipio = m.
        codigo_municipio_dv
25 LEFT JOIN unidade_federacao u ON m.cd_uf         = u.cd_uf
26 LEFT JOIN regiao            r ON u.cd_regiao      = r.cd_regiao;
```

### 3.6.1 Visão Física x Visão Lógica

O usuário de banco disponibilizado (data\_iesb) possui acesso somente leitura: a tentativa de criar a visão física resultou em `permission denied for schema public`. Para contornar a limitação sem perder funcionalidade, o sistema opera por padrão em modo **visão lógica**: a mesma definição do Código 3.1 é utilizada como subconsulta,

---

mantendo o comportamento 100% somente leitura. Caso o projeto seja executado com um usuário que tenha permissão de escrita, a visão física é criada automaticamente. Essa decisão, além de pragmática, reforça a postura de segurança do sistema.

## 4 DESENVOLVIMENTO E ARQUITETURA

### 4.1 ARQUITETURA DO SISTEMA

A Figura 1 ilustra o fluxo. A interface (Streamlit) envia a pergunta ao módulo de orquestração (`chatbot.py`), que aciona o Bedrock (`bedrock_client.py`) para obter a intenção em JSON e, então, chama a função segura correspondente em `database.py`. O resultado é formatado e devolvido ao usuário.

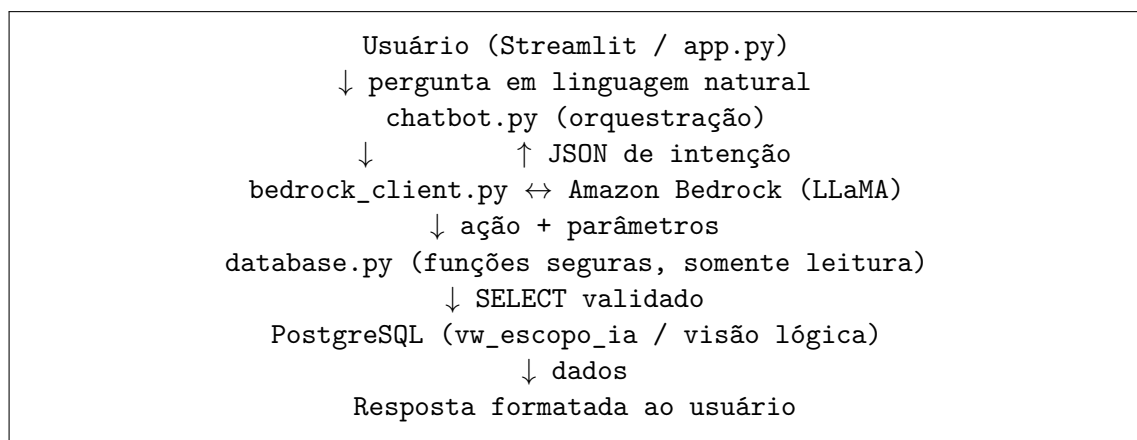


Figura 1 – Arquitetura e fluxo de dados do chatbot.

### 4.2 TECNOLOGIAS UTILIZADAS

- Python 3.12;
- Streamlit (Streamlit Inc., 2025) — interface web;
- Amazon Bedrock (Amazon Web Services, 2025b) via boto3 (Amazon Web Services, 2025c);
- PostgreSQL 17 (The PostgreSQL Global Development Group, 2025) com SQLAlchemy (BAYER, 2025) e psycopg2;
- pandas (exibição de tabelas) e python-dotenv (configuração).

### 4.3 INTEGRAÇÃO COM O AMAZON BEDROCK

A autenticação usa o IAM Identity Center (SSO): no primeiro uso, o navegador abre para login e as credenciais temporárias ficam em cache (cerca de 8 horas). A região do `bedrock-runtime` é configurável (padrão `us-east-2`); o login SSO ocorre em `us-east-1`.

O *prompt* de sistema instrui o modelo a retornar apenas JSON, escolhendo uma ação de uma lista fixa (Código 4.1).

Listing 4.1 – Trecho do *prompt* de sistema (classificador de intenções).

```
1 Você é um classificador de intenções para um chatbot acadêmico
   conectado a um
2 banco de dados educacional do INEP/Censo Escolar.
3 Sua ÚNICA função é interpretar a pergunta e retornar um JSON vá
   lido.
4 NÃO gere SQL. NÃO explique. Escolha UMA ação da lista permitida.
5 Quando o usuário falar em "a base", "censo escolar", "INEP" ou "
   matrículas",
6 considere a base final vw_escopo_ia.
7
8 Formato de saída:
9 { "acao": "...", "tabela": "...", "coluna": "...", "filtro":
   "...", "limite": null }
```

Para a pergunta “*Quantas matrículas existem no ensino fundamental por UF?*”, o modelo retorna:

```
1 { "acao": "total_matriculas_por_uf", "tabela": "vw_escopo_ia",
2   "coluna": "QT_MAT_FUND", "filtro": null, "limite": null }
```

## 4.4 CONEXÃO COM O BANCO DE DADOS

A conexão é feita via SQLAlchemy, montando a URL a partir de variáveis de ambiente (arquivo `.env`), o que mantém as credenciais fora do código. O sistema é agnóstico ao SGBD (PostgreSQL, MySQL ou SQLite). A introspecção utiliza o *inspector* do SQLAlchemy para listar tabelas, colunas, chaves primárias e estrangeiras.

## 4.5 FUNÇÕES IMPLEMENTADAS

As funções de metadados (`database.py`) incluem `conectar_banco`, `listar_tabelas`, `contar_colunas`, `listar_colunas`, `contar_registros`, `descrever_tabela`, `mostrar_amostra`, `identificar_chave_primaria`, `identificar_relacionamentos`, `validar_tabela` e `validar_coluna`. As funções do escopo educacional incluem `inspecionar_tabelas_escopo`, `identificar_chaves_relacionamentos_escopo`, `criar_ou_validar_view_escopo_ia`, `listar_variaveis_escopo`, as consultas de total de matrículas por município/UF/região, por etapa de ensino, por gênero, por raça/cor, a distribuição por localização, os valores distintos e o resumo estatístico.

## 4.6 CONSULTA LIVRE EM LINGUAGEM NATURAL (NL→SQL)

Além das ações estruturadas (determinísticas) do escopo, o sistema possui um modo **NL→SQL** que permite responder **qualquer** pergunta sobre os **dados das tabelas do escopo**. Quando a pergunta não corresponde a uma ação pré-definida, o classificador retorna a ação `consulta_sql` e o fluxo passa a ser: (i) o Bedrock recebe o esquema da base consolidada `vw_escopo_ia` e gera um comando `SELECT`; (ii) o Python valida que se trata de uma consulta somente leitura e que ela referencia apenas tabelas do escopo (*allowlist*); (iii) como a *view* é lógica, ela é injetada como CTE e a consulta é executada; (iv) o Bedrock resume o resultado em linguagem natural. Assim, perguntas como “*Qual a média de matrículas do ensino médio por município em Goiás?*” são respondidas dinamicamente, enquanto perguntas fora do escopo (ex.: PIB, acidentes) são recusadas. Esse modo pode ser desativado (`PERMITIR_SQL_LIVRE=false`) ou ampliado para todo o banco (`PERMITIR_TODAS_TABELAS=true`) via variáveis de ambiente.

## 4.7 FLUXO DE FUNCIONAMENTO DO CHATBOT

1. O usuário digita a pergunta na interface.
2. `chatbot.py` envia a pergunta ao Bedrock.
3. O Bedrock devolve o JSON de intenção (ação + parâmetros).
4. O JSON é normalizado e validado contra a lista de ações permitidas.
5. A função segura correspondente em `database.py` é chamada.
6. O resultado é formatado (texto + tabela) e exibido ao usuário.

## 5 RESULTADOS E DISCUSSÃO

### 5.1 PRINTS DA EXECUÇÃO

Os espaços a seguir são reservados para capturas de tela da execução (`streamlit run app.py`).

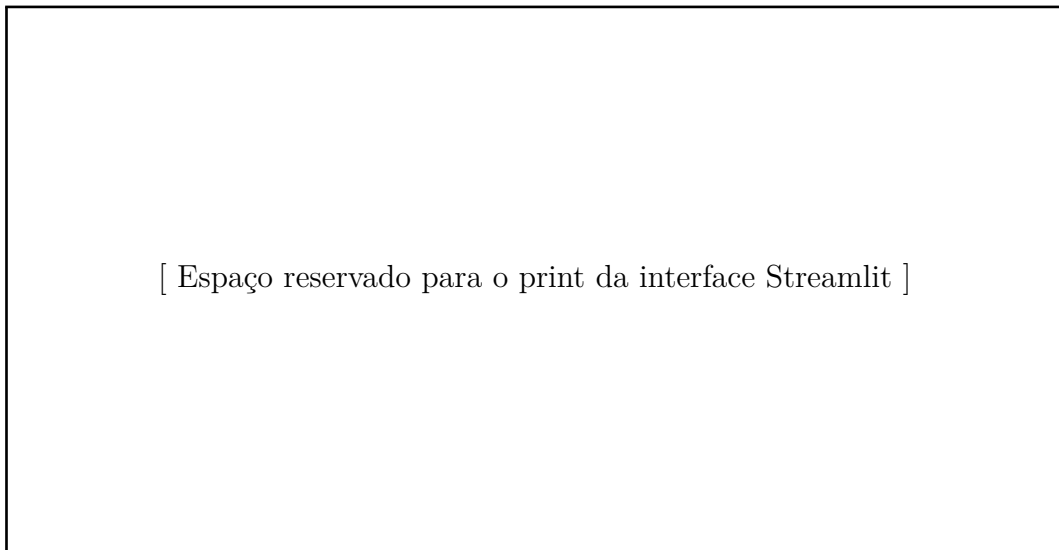


Figura 2 – Tela principal do chatbot.

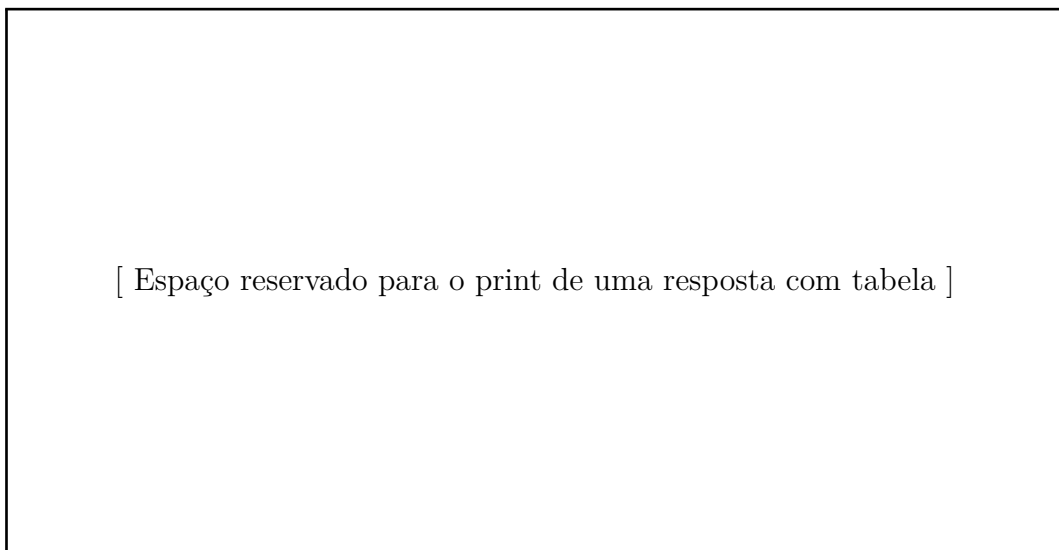


Figura 3 – Exemplo de resposta (matrículas por UF).



## 5.2 TESTES REALIZADOS

Foram implementados testes automatizados com `pytest` (arquivo `tests/test_database.py`), divididos em: (i) testes de segurança (*offline*), que verificam o bloqueio de comandos destrutivos e a sanitização de parâmetros; e (ii) testes de integração, que validam a conexão real, a presença das variáveis do escopo e a coerência das agregações. Na execução, todos os 17 testes passaram. Adicionalmente, validou-se o fluxo completo (Bedrock + banco) com perguntas reais.

## 5.3 EXEMPLOS DE PERGUNTAS E RESPOSTAS

A Tabela 3 apresenta perguntas reais e os valores retornados (Censo 2025).

Tabela 3 – Exemplos de perguntas e respostas obtidas com dados reais.

| Pergunta                            | Resposta (resumo)                                                                                                       |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Qual região possui mais matrículas? | Sudeste: 16.910.475; Nordeste: 11.768.637; Sul: 5.974.917; Norte: 4.340.431; Centro-Oeste: 3.469.368.                   |
| Matrículas femininas e masculinas?  | Feminino: 22.735.128; Masculino: 23.283.252.                                                                            |
| Matrículas por raça/cor?            | Parda: 20.847.435; Branca: 16.044.865; Não declarada: 6.275.577; Preta: 2.301.626; Indígena: 381.498; Amarela: 167.379. |
| Distribuição por localização?       | Urbana: 144.217 escolas / 37.869.396 matrículas; Rural: 69.975 escolas / 4.594.432 matrículas.                          |
| Quantas colunas tem a base?         | A base <code>vw_escopo_ia</code> possui 18 colunas.                                                                     |

## 5.4 DIFICULDADES ENCONTRADAS

- **Variáveis em duas tabelas:** as `QT_MAT_*` não estavam na tabela principal, e sim em `inep_censo_escolar_matricula`; resolvido incorporando essa tabela à junção.
- **Acesso somente leitura:** não foi possível criar a visão física; adotou-se a visão lógica (subconsulta equivalente).
- **Saída do LLM:** modelos abertos podem incluir texto extra junto ao JSON; implementou-se extração robusta do primeiro bloco JSON balanceado e *fallback* para pergunta\_nao\_suportada.

- **Maiúsculas/minúsculas:** divergência entre o escopo (nomes em maiúsculas) e o banco (minúsculas), tratada por resolução *case-insensitive* e *quoting* de identificadores.
- **Valores já textuais:** colunas como TP\_LOCALIZACAO retornam “Urbana”/“Rural” (texto), exigindo rotulagem condicional.

## 5.5 RESULTADOS OBTIDOS

O chatbot respondeu corretamente, com dados reais, a perguntas de metadados e do escopo educacional. A base consolidada possui 18 colunas e abrange 214.192 escolas (Censo 2025). As agregações por região, UF, etapa, gênero e raça/cor foram validadas. A separação entre interpretação (IA) e execução (código) mostrou-se eficaz, segura e de fácil manutenção.

## 5.6 LIMITAÇÕES DO PROJETO

- O conjunto de perguntas é limitado às ações pré-definidas (não há SQL livre).
- Filtros compostos (ex.: “matrículas no Sudeste em escolas rurais”) ainda são limitados.
- Depende de credenciais válidas de AWS/SSO e de conectividade com o banco remoto.
- A qualidade da classificação depende do modelo de fundação utilizado.

## 5.7 POSSIBILIDADES DE MELHORIA

- Suporte a filtros combinados (UF + localização + etapa) na intenção.
- Geração de gráficos automáticos na interface.
- Cache de respostas e memória de contexto conversacional.
- Avaliação quantitativa da acurácia de classificação de intenções.
- Exportação de resultados (CSV/Excel) e *logging* de auditoria.

## 6 CONCLUSÃO

O projeto atingiu os objetivos propostos: um chatbot funcional, integrado ao Amazon Bedrock e conectado a um banco PostgreSQL real, capaz de responder em linguagem natural a perguntas sobre o INEP/Censo Escolar, dentro do escopo definido pelo docente. A arquitetura adotada — IA apenas para interpretar a intenção e código auditável para executar consultas seguras — equilibra usabilidade e segurança, constituindo um padrão replicável para integração entre LLMs e bancos de dados. As dificuldades reais encontradas (variáveis em duas tabelas e acesso somente leitura) foram contornadas de forma documentada, reforçando o caráter prático e seguro da solução.

# REFERÊNCIAS

- Amazon Web Services. *Amazon Bedrock - Converse API*. 2025. <<https://docs.aws.amazon.com/bedrock/latest/userguide/converseapi.html>>. Acesso em: jun. 2026.
- Amazon Web Services. *Amazon Bedrock Documentation*. 2025. <<https://docs.aws.amazon.com/bedrock/>>. Acesso em: jun. 2026.
- Amazon Web Services. *Boto3 - AWS SDK for Python*. 2025. <<https://boto3.amazonaws.com/v1/documentation/api/latest/index.html>>. Acesso em: jun. 2026.
- BAYER, M. *SQLAlchemy - The Database Toolkit for Python*. 2025. <<https://www.sqlalchemy.org/>>. Acesso em: jun. 2026.
- Instituto Nacional de Estudos e Pesquisas Educacionais Anísio Teixeira (INEP). *Censo Escolar da Educação Básica*. 2025. <<https://www.gov.br/inep/pt-br/areas-de-atuacao/pesquisas-estatisticas-e-indicadores/censo-escolar>>. Acesso em: jun. 2026.
- Meta AI. *Llama: Open Foundation Models*. 2025. <<https://ai.meta.com/llama/>>. Acesso em: jun. 2026.
- OWASP Foundation. *OWASP Top 10 - 2021: A03 Injection*. [S.l.: s.n.], 2021. <[https://owasp.org/Top10/A03\\_2021-Injection/](https://owasp.org/Top10/A03_2021-Injection/)>. Acesso em: jun. 2026.
- Streamlit Inc. *Streamlit Documentation*. 2025. <<https://docs.streamlit.io/>>. Acesso em: jun. 2026.
- The PostgreSQL Global Development Group. *PostgreSQL 17 Documentation*. 2025. <<https://www.postgresql.org/docs/>>. Acesso em: jun. 2026.
- VASWANI, A. et al. Attention is all you need. *Advances in Neural Information Processing Systems*, v. 30, 2017.
- YU, T. et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. [S.l.: s.n.], 2018.
- ZHONG, V.; XIONG, C.; SOCHER, R. Seq2sql: Generating structured queries from natural language using reinforcement learning. *arXiv preprint arXiv:1709.00103*, 2017.

## Apêndices

# APÊNDICE A – TRECHOS PRINCIPAIS DO CÓDIGO

## A.1 DESCOBERTA DE RELACIONAMENTOS (SEM INVENTAR CHAVES)

```

1 def _achar_fk(insp, tabela, alvo_lower):
2     """Retorna a primeira FK de tabela que referencia a tabela
3     alvo."""
4     for fk in insp.get_foreign_keys(tabela, schema=config.
5     DB_SCHEMA):
6         if (fk.get("referred_table") or "").lower() == alvo_lower
7             :
8             return fk
9     return None

```

## A.2 GUARDA DE SEGURANÇA (SOMENTE LEITURA)

```

1 _DESTRUTIVO = re.compile(
2     r"\b(DROP|DELETE|UPDATE|INSERT|ALTER|TRUNCATE|GRANT|REVOKE|
3     CREATE|
4     REPLACE|MERGE|CALL|EXECUTE|COPY)\b", re.IGNORECASE)
5
6 def _assert_select_only(sql):
7     limpo = sql.strip().lstrip("(").strip()
8     if not re.match(r"^(SELECT|WITH)\b", limpo, re.IGNORECASE):
9         raise BancoError("Consulta bloqueada: apenas SELECT e
10         permitido.")
11     if _DESTRUTIVO.search(sql):
12         raise BancoError("Consulta bloqueada: comando destrutivo
13         detectado.")

```

## A.3 AGREGAÇÃO SEGURA DE MATRÍCULAS POR GRUPO

---

```
1 def _agrupar_matriculas(coluna_grupo, limite=None, coluna_etapa=
   None):
2     fonte = _fonte_escopo()                # visao fisica ou
   subconsulta logica
3     grupo_q = _quote(_colunas_existentes([coluna_grupo])[0])
4     medida = (f"SUM(COALESCE({_quote(coluna_etapa)},0))" if
   coluna_etapa
5               else f"SUM({_soma_total_matriculas_sql()})")
6     sql = (f"SELECT {grupo_q} AS grupo, {medida} AS
   total_matriculas "
7           f"FROM {fonte} WHERE {grupo_q} IS NOT NULL "
8           f"GROUP BY {grupo_q} ORDER BY total_matriculas DESC
   NULLS LAST")
9     # ... LIMIT parametrizado e execucao somente leitura ...
```